GIÁO TRÌNH ĐÀO TẠO KỸ SƯ FRONTEND CHUYÊN SÂU

GIÁO TRÌNH CSS3

TẬP 2: BỐ CỤC HIỆN ĐẠI & NÂNG CAO

Chuyên Đề Box Model, Outline, Flexbox, CSS Grid, Responsive & Animations

■ CSS Box Model • ■ Flexbox & Grid • ■ Responsive Web • ■ Animations

TÀI LIỆU CHUYÊN SÂU TẬP TRUNG CSS3 – TẬP 2

Làm Chủ Mô Hình Hộp (Box Model) • Bố Cục Một Chiều Flexbox & Hai Chiều CSS Grid

Hệ thống hóa toàn diện phương pháp dàn trang hiện đại • Thiết kế Responsive thích ứng đa màn hình • Tối ưu hiệu năng render chuẩn kỹ sư UI/UX

ĐỘI NGŨ BIÊN SOẠN WEB

Tài liệu lưu hành nội bộ

NĂM 2026

Phiên bản Standalone Tập 2

ĐỘI NGŨ BIÊN SOẠN WEB

TÀI LIỆU LƯU HÀNH NỘI BỘ – CHUYÊN ĐỀ CSS3 TẬP 2

GIÁO TRÌNH CSS3

TẬP 2: BỐ CỤC HIỆN ĐẠI & NÂNG CAO

Box Model, Outline, Flexbox, Grid, Responsive & Animations

TẬP TRUNG BIÊN SOẠN & THỰC HÀNH CSS3 NÂNG CAO

Biên soạn chi tiết • Minh họa trực quan • Sẵn sàng tích hợp nội dung mới

NĂM 2026

Phiên bản Standalone Tập 2 – XeLaTeX Unicode

MỤC LỤC

Giáo Trình Chuyên Sâu CSS3 – Tập 2: Bố Cục Hiện Đại & Kỹ Thuật Chuyên Sâu

I CSS3 Nâng Cao – Mô Hình Hộp & Bố Cục Hiện Đại 1

Chương 1: Chuyên Đề Toàn Diện: CSS Box Model & Thuộc Tính Outline 2

1.1 Tổng Quan Thuộc Tính `width` & `height` Trong CSS 2

1.1.1 1. Thuộc Tính `width` (Chiều Rộng) 2

1.1.2 2. Thuộc Tính `height` (Chiều Cao) 4

1.1.3 3. Các Thuộc Tính Liên Quan và Quan Trọng 5

1.1.4 4. Khi Nào Nên Dùng auto Và Khi Nào Nên Dùng Giá Trị Cố Định? 7

1.2 Cách Hoạt Động Của `width` / `height` & CSS Box Model 8

1.2.1 1. CSS Box Model (Mô Hình Hộp) 8

1.2.2 2. Thuộc Tính `box-sizing` (Thay Đổi Cách Tính Toán Kích Thước) 9

1.3 Lưu Ý Chi Tiết Khi Sử Dụng `width` & `height` 10

1.3.1 1. Giá Trị auto Và Hành Vi Khác Biệt Giữa Các Loại Phần Tử 11

1.3.2 2. Đơn Vị Tương Đối Và Đơn Vị Tuyệt Đối 12

1.3.3 3. Kiểm Soát Giới Hạn: `min-width`, `max-width`, `min-height`, `max-height` 12

1.3.4 4. Mối Tương Quan & Ảnh Hưởng Từ `display` Và `position` 13

1.3.5 5. Tương Tác Giữa `overflow` Và Kích Thước `width` / `height` 14

1.3.6 6. Lời Khuyên Quan Trọng & Thiết Lập Reset Toàn Cục Chuẩn 14

Phần I

CSS3 Nâng Cao – Mô Hình Hộp & Bố Cục Hiện Đại

CHƯƠNG 1

Chuyên Đề Toàn Diện: CSS Box Model & Thuộc Tính Outline

[Khái Niệm] Khởi Tạo Tài Liệu CSS Tập 2

Tập 2 của bộ giáo trình CSS3 chuyên sâu tập trung vào các chủ đề nâng cao và kiến trúc giao diện hiện đại:

- **Kích thước & Mô hình hộp (CSS Sizing & Box Model)**: Bản chất `width`, `height`, cấu trúc 4 lớp, thuật toán `box-sizing`, hiện tượng sập lề (**margin collapsing**) và ranh giới `outline` vs `border`.
- **Bố cục một chiều (CSS Flexbox)**: Trục chính, trục phụ, phân bổ không gian và kỹ thuật căn chỉnh hoàn hảo.
- **Bố cục hai chiều (CSS Grid)**: Lưới đa chiều, vùng lưới (**grid-template-areas**), đơn vị phân số `fr` và thuật toán tự động co giãn `auto-fit` / `auto-fill`.
- **Thiết kế thích ứng (Responsive Design)**: Tư duy Mobile-First, Container Queries, Fluid Typography với `clamp()`.
- **Chuyển động mượt mà (Transitions & Keyframe Animations)**: Tương tác vi mô (**micro-interactions**) và hoạt họa UI sinh động.

1.1 Tổng Quan Thuộc Tính `width` & `height` Trong CSS

`width` và `height` là hai thuộc tính CSS cơ bản và cực kỳ quan trọng, dùng để thiết lập chiều rộng và chiều cao của một phần tử HTML. Chúng là nền tảng để kiểm soát kích thước và bố cục của mọi thứ trên trang web của bạn.

[Khái Niệm] Bản Chất Của width & height Trong Thiết Kế Web

Mọi thành phần hiển thị trên trang web (khung bài viết, nút bấm, hình ảnh, thanh điều hướng, ô nhập liệu) đều cần được định hình không gian rõ ràng. `width` và `height` đóng vai trò là chiếc thước đo cơ bản nhất để lập trình viên xác định diện tích mà một khối nội dung sẽ chiếm giữ trên mặt phẳng giao diện hai chiều.

1.1.1 1. Thuộc Tính `width` (Chiều Rộng)

Thuộc tính `width` xác định chiều rộng của vùng nội dung (**content area**) của một phần tử.

Các Giá Trị Phổ Biến Của width

1. `auto` **(Mặc định)**:
   - **Mô tả**: Trình duyệt sẽ tự động tính toán chiều rộng.
   - **Hành vi chi tiết**:
     - Đối với các phần tử khối (**block-level** như `<div>`, `<p>`, `<h1>`–`<h6>`): `width: auto;` thường khiến phần tử chiếm toàn bộ chiều rộng có sẵn của phần tử cha nó, trừ đi `margin`, `padding`, và `border`.
     - Đối với các phần tử nội dòng khối (**inline-block**) hoặc phần tử trôi nổi (**float**): `width: auto;` sẽ khiến chiều rộng co lại (**shrink-wrap**) để vừa khít với nội dung bên trong nó.
   - **Khi sử dụng**: Khi bạn muốn trình duyệt tự quản lý kích thước dựa trên nội dung hoặc không gian có sẵn.
2. `length` **(Giá trị cụ thể)**:
   - **Mô tả**: Đặt một chiều rộng cố định bằng các đơn vị đo lường CSS.
   - **Hệ thống đơn vị phổ biến**:
     - `px` (**pixels**): Đơn vị cố định, không thay đổi theo môi trường hay tỷ lệ phông chữ. Ví dụ: `width: 200px;`.
     - `em`: Tương đối so với kích thước phông chữ (`font-size`) của phần tử cha (hoặc chính nó nếu có khai báo `font-size`). Ví dụ: `width: 10em;` (nếu `font-size` của phần tử cha là 16px, thì $10\text{em} = 10 \times 16\text{px} = 160\text{px}$).
     - `rem`: Tương đối so với kích thước phông chữ gốc của tài liệu (thẻ `<html>`, mặc định 16px). Ví dụ: `width: 10rem;` (tương đương $10 \times 16\text{px} = 160\text{px}$).
     - `vw` (**viewport width**): Tương đối so với chiều rộng của khung nhìn trình duyệt (**viewport**). $1\text{vw} = 1\%$ chiều rộng viewport. Ví dụ: `width: 50vw;` (chiếm chính xác 50% chiều rộng màn hình thiết bị).
     - `%` (**percentage**): Tương đối so với chiều rộng của phần tử cha trực tiếp. Ví dụ: `width: 80%;` (chiếm 80% chiều rộng vùng đệm/nội dung của cha).
   - **Khi sử dụng**: Khi bạn cần kiểm soát chính xác kích thước theo bản vẽ thiết kế (UI/UX wireframe) hoặc yêu cầu bố cục nghiêm ngặt.
3. `initial`: Đặt thuộc tính về giá trị mặc định của nó theo đặc tả W3C (`auto`).
4. `inherit`: Kế thừa giá trị chiều rộng cụ thể từ phần tử cha trực tiếp.

Minh Họa Khai Báo Các Giá Trị Của Thuộc Tính width

```css
/* 1. Gia tri mac dinh tu dong tinh toan */
.card-auto {
    width: auto;
}

/* 2. Don vi tuyet doi pixel */
.sidebar {
    width: 280px;
}

/* 3. Don vi tuong doi theo font chu */
.text-column-em {
    width: 25em;   /* Co gian theo co chu */
}

.text-column-rem {
    width: 40rem;  /* Co gian theo co chu goc html */
}

/* 4. Don vi phan tram va viewport */
.main-content {
    width: 75%;    /* Chiay 75% be rong phan tu cha */
}

.hero-banner {
    width: 100vw;  /* Phu kin 100% chieu rong man hinh */
}
```

1.1.2 2. Thuộc Tính `height` (Chiều Cao)

Thuộc tính `height` xác định chiều cao của vùng nội dung (**content area**) của một phần tử.

Các Giá Trị Phổ Biến Của height

1. `auto` **(Mặc định)**:
   - **Mô tả**: Trình duyệt sẽ tự động tính toán chiều cao dựa trên dòng chảy nội dung bên trong.
   - **Hành vi chi tiết**: Chiều cao của phần tử sẽ co lại hoặc giãn ra linh hoạt để vừa khít với toàn bộ lượng nội dung bên trong nó (chữ, hình ảnh, khối con).
   - **Khi sử dụng**: Đây là hành vi mong muốn trong hầu hết các trường hợp xây dựng trang web, giúp nội dung không bao giờ bị cắt cụt và phần tử tự động thích ứng với mọi độ dài văn bản hoặc kích thước màn hình.
2. `length` **(Giá trị cụ thể)**:
   - **Mô tả**: Đặt một chiều cao cố định bằng các đơn vị đo lường CSS.
   - **Hệ thống đơn vị**: Tương tự như `width`, bao gồm `px`, `em`, `rem`, `vh` (**viewport height**), và `%`:

  - `vh` (**viewport height**): Tương đối so với chiều cao của khung nhìn trình duyệt. 1vh = 1% chiều cao viewport. Ví dụ: `height: 100vh;` (chiếm trọn vẹn 100% chiều cao màn hình trình duyệt).
- **Khi sử dụng**: Khi bạn cần một chiều cao chính xác, ví dụ: tạo một banner cố định, ảnh avatar, thanh điều hướng (**header bar**), hoặc các phần tử trong bố cục dạng lưới (**grid layout**) hoặc **flexbox**.

3. `initial`: Đặt thuộc tính về giá trị mặc định của nó theo quy chuẩn (`auto`).
4. `inherit`: Kế thừa giá trị chiều cao từ phần tử cha trực tiếp.

Minh Họa Khai Báo Các Giá Trị Của Thuộc Tính height

```css
/* 1. Tu dong co gian theo noi dung (Khuyen nghi cho khoi chua van ban) */
.article-body {
    height: auto;
}

/* 2. Chieu cao co dinh cho thanh tieu de (Navigation Bar) */
.navbar {
    height: 64px;
}

/* 3. Man hinh chao phu tron 100% chieu cao viewport */
.hero-fullscreen {
    height: 100vh;
}

/* 4. Khung anh vuong ti le dong */
.avatar-thumb {
    width: 80px;
    height: 80px;
}
```

1.1.3 3. Các Thuộc Tính Liên Quan và Quan Trọng

Để kiểm soát kích thước một cách linh hoạt, an toàn và chống vỡ giao diện trên nhiều thiết bị, bạn sẽ thường xuyên sử dụng kết hợp các thuộc tính sau:

[Bảng] Bảng Tra Cứu Các Thuộc Tính Kiểm Soát Kích Thước Liên Quan

Thuộc tính	Mục đích & Hành vi cốt lõi	Cú pháp mẫu
`min-width` `min-height`	Thiết lập ngưỡng kích thước tối thiểu. Phần tử không thể co nhỏ hơn ngưỡng này dù không gian bị thu hẹp.	`min-width: 300px;` `min-height: 40px;`
`max-width` `max-height`	Thiết lập ngưỡng kích thước tối đa. Phần tử không thể phình to hơn ngưỡng này dù màn hình rất rộng.	`max-width: 1200px;` `max-height: 300px;`
`box-sizing`	Quyết định thuật toán tính `width`/`height` có gộp cả `padding` và `border` vào hay không.	`box-sizing: border-box;`
`overflow`	Xử lý hành vi hiển thị khi nội dung bên trong vượt quá kích thước `width`/`height` đã định.	`overflow: auto;`

Chi Tiết Từng Thuộc Tính Bổ Trợ

1. `min-width` / `min-height`:
   - Đặt chiều rộng/chiều cao tối thiểu mà một phần tử có thể có. Phần tử sẽ không nhỏ hơn giá trị này, ngay cả khi không gian bị hạn chế.
   - Rất hữu ích trong thiết kế đáp ứng (**responsive design**) để ngăn phần tử bị quá nhỏ dẫn đến méo mó hoặc không đọc được chữ.
   - **Ví dụ**: `min-width: 300px;`.
2. `max-width` / `max-height`:
   - Đặt chiều rộng/chiều cao tối đa mà một phần tử có thể có. Phần tử sẽ không lớn hơn giá trị này, ngay cả khi có nhiều không gian trống.
   - Cũng rất hữu ích trong thiết kế đáp ứng, ví dụ: `max-width: 100%;` cho hình ảnh để chúng tự co lại linh hoạt và không bao giờ tràn ra ngoài phần tử cha.
   - **Ví dụ**: `max-width: 600px;`.
3. `box-sizing`:
   - Đây là một thuộc tính cực kỳ quan trọng ảnh hưởng trực tiếp đến cách `width` và `height` được tính toán.
   - `content-box` (**Mặc định**): `width` và `height` chỉ áp dụng cho vùng nội dung của phần tử. `padding` và `border` được thêm vào bên ngoài kích thước này, làm cho phần tử thực tế lớn hơn so với con số khai báo.
   - `border-box`: `width` và `height` bao gồm cả `padding` và `border`. Đây là mô hình hộp trực quan hơn và dễ làm việc hơn trong bố cục. Nó được khuyến nghị sử dụng trong hầu hết các dự án hiện đại.
4. `overflow`:
   - Kiểm soát những gì xảy ra khi nội dung của một phần tử tràn ra khỏi kích thước `width` và `height` đã định.

- Các giá trị phổ biến:
  - `visible` (Mặc định): Nội dung tràn hẳn ra ngoài phần tử và đè lên các thành phần xung quanh.
  - `hidden`: Cắt bỏ phần nội dung thừa vượt ra ngoài khung chứa và ẩn nó đi.
  - `scroll`: Luôn luôn hiển thị thanh cuộn (cả ngang lẫn dọc), cho phép người dùng cuộn xem phần bị tràn.
  - `auto`: Chỉ hiển thị thanh cuộn khi nội dung thực sự vượt quá kích thước khung chứa.

Ví Dụ So Sánh Kích Thước Với box-sizing: content-box vs border-box

```
/* Ap dung cho tat ca cac phan tu de thong nhat cach tinh toan */
* {
    box-sizing: border-box;
}

.box {
    width: 200px;
    padding: 10px;
    border: 2px solid black;
    /* Voi border-box, chieu rong tong cong van la dung 200px. */
    /* Voi content-box, chieu rong tong cong se la: 200 + 10*2 + 2*2 = 224px. */
}
```

1.1.4 4. Khi Nào Nên Dùng auto Và Khi Nào Nên Dùng Giá Trị Cố Định?

1. **Khi nên sử dụng `width: auto;` và `height: auto;`:**
   - **Trong hầu hết các trường hợp, đặc biệt là với `height`**: Bạn thường muốn phần tử tự động điều chỉnh chiều cao theo lượng nội dung để tránh nội dung bị cắt cụt (**clipping**) khi văn bản dài ra hoặc đổi ngôn ngữ.
   - **Khi bạn muốn phần tử khối (block) chiếm toàn bộ chiều rộng có sẵn**: Tận dụng cơ chế lấp đầy tự nhiên của trình duyệt mà không cần khai báo cứng `width: 100%`.
   - **Khi bạn muốn phần tử `inline-block` co lại vừa với nội dung**: Tạo các nút bấm (**button**), nhãn (**badge**) hay thẻ (**tag**) vừa vặn với chiều dài từ ngữ bên trong.
2. **Khi nên sử dụng `width` và `height` với giá trị cụ thể (`px`, `%`, `vw`, `vh`…)**:
   - Khi bạn cần kiểm soát kích thước chính xác cho một thiết kế cụ thể (ví dụ: kích thước logo, hình ảnh đại diện avatar, khung chứa cố định).
   - Để tạo các cột có chiều rộng nhất định (ví dụ: sidebar rộng 280px) hoặc các vùng có chiều cao tối thiểu/tối đa.
   - Khi làm việc với mô hình bố cục hiện đại như **Flexbox** hoặc **CSS Grid** để định hình các phần tử con (**flex/grid items**).

[Khái Niệm] Viên Gạch Đầu Tiên Của Bố Cục Giao Diện Web

`width` và `height` là những viên gạch đầu tiên trong việc xây dựng bố cục web. Kết hợp chúng một cách thông minh với các thuộc tính liên quan như `min-width`, `max-width`, `box-sizing`, và `overflow` sẽ giúp bạn tạo ra những giao diện linh hoạt, đáp ứng (**responsive**) và đẹp mắt trên mọi thiết bị.

1.2 Cách Hoạt Động Của `width` / `height` & CSS Box Model

Khi bạn đặt `width` và `height` cho một phần tử trong CSS, bạn đang thiết lập kích thước của **vùng nội dung (content area)** của phần tử đó. Tuy nhiên, kích thước thực tế mà phần tử chiếm trên trang (gọi là **layout width/height**) lại phức tạp hơn một chút, vì nó còn bị ảnh hưởng bởi `padding`, `border`, và `margin`. Đây chính là lúc **CSS Box Model (Mô hình hộp)** phát huy vai trò quyết định.

1.2.1 1. CSS Box Model (Mô Hình Hộp)

Mọi phần tử HTML trên trang web đều được xem như một chiếc hộp hình chữ nhật, bao gồm bốn lớp xếp chồng lên nhau từ trong ra ngoài:

1. **Content (Nội dung)**: Vùng chứa nội dung thực tế của phần tử (văn bản, hình ảnh, video, danh sách). Đây chính là vùng mà `width` và `height` trực tiếp tác động khi sử dụng mô hình chuẩn.
2. **Padding (Vùng đệm)**: Khoảng không gian đệm trong suốt nằm giữa nội dung và đường viền (`border`) của phần tử. `padding` có màu nền trùng với màu nền của phần tử.
3. **Border (Đường viền)**: Đường viền bao bọc xung quanh vùng đệm (`padding`) và nội dung (`content`). Đường viền có thể được định dạng độ dày, màu sắc và kiểu nét vẽ.
4. **Margin (Lề ngoài)**: Khoảng không gian trống hoàn toàn trong suốt bên ngoài đường viền, dùng để tạo khoảng cách tách biệt giữa phần tử này với các phần tử khác xung quanh.

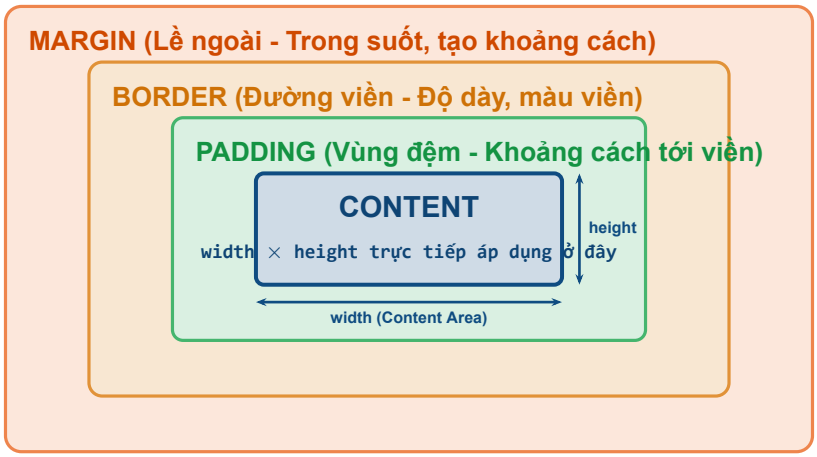
Sơ Đồ Hình Học CSS Box Model Phân Tầng

Dưới đây là sơ đồ ASCII nguyên bản thể hiện cấu trúc lồng nhau của 4 lớp Box Model:

Sơ Đồ Mô Phỏng 4 Lớp Mô Hình Hộp (ASCII Diagram)

```
+-----------------------------------------+
|                 Margin                  |
|   +---------------------------------+   |
|   |              Border             |   |
|   |   +-------------------------+   |   |
|   |   |         Padding         |   |   |
|   |   |   +-----------------+   |   |   |
|   |   |   |     Content     |   |   |   | <-- width & height áp dụng ở đây
|   |   |   +-----------------+   |   |   |
|   |   +-------------------------+   |   |
|   +---------------------------------+   |
+-----------------------------------------+
```

BROWSER PREVIEW **Sơ Đồ Trực Quan Hóa 4 Lớp CSS Box Model (Chuẩn Browser DevTools)**



1.2.2 2. Thuộc Tính `box-sizing` (Thay Đổi Cách Tính Toán Kích Thước)

Đây là thuộc tính quan trọng nhất ảnh hưởng trực tiếp đến cách trình duyệt tính toán `width` và `height` thực tế của phần tử trên trang.

Mô Thức 1: `box-sizing: content-box;` (Mặc Định Chuẩn W3C)

- `width` và `height` chỉ bao gồm kích thước của **Content Area (Vùng nội dung)**.
- `padding` và `border` sẽ được **cộng thêm vào bên ngoài** kích thước đã đặt của `width` và `height`.
- **Ví dụ tính toán thực tế**: Nếu bạn có một thẻ `<div>` với:
  - Khai báo: `width: 200px; padding: 10px; border: 2px solid;`
  - Chiều rộng nội dung là: 200px.

- Chiều rộng thực tế mà phần tử chiếm trên trang (**layout width**) sẽ là:

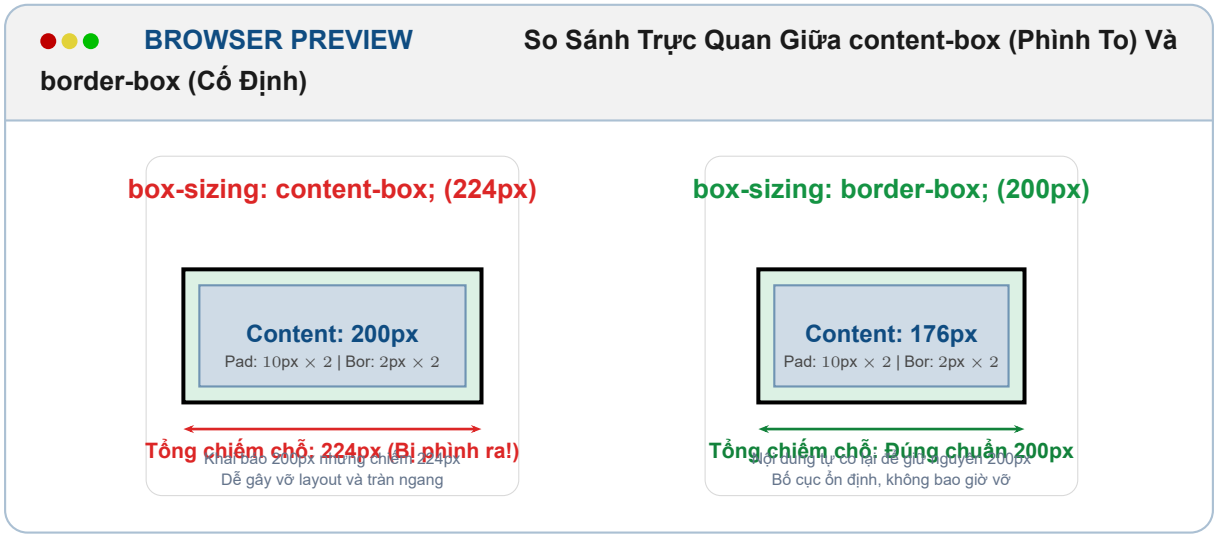
$$\text{Layout Width} = 200\text{px (content)} + 10\text{px (left pad)} + 10\text{px (right pad)} + 2\text{px (left bor)} + 2\text{px (right bor)} = \mathbf{224}\text{px}$$

Mô Thức 2: `box-sizing: border-box;` (Rất Được Khuyến Nghị)

- `width` và `height` bao gồm toàn bộ kích thước của **Content Area + Padding + Border**.
- `padding` và `border` sẽ được **tính gộp vào bên trong** kích thước `width` và `height` đã đặt. Vùng nội dung (Content Area) sẽ tự động co lại để nhường chỗ cho `padding` và `border`.
- **Ví dụ tính toán thực tế**: Nếu bạn có một thẻ `<div>` với:
  - Khai báo: `width: 200px; padding: 10px; border: 2px solid; box-sizing: border-box;`
  - Chiều rộng thực tế mà phần tử chiếm trên trang (**layout width**) sẽ là: đúng **200**px.
  - Chiều rộng nội dung còn lại sẽ là:

$$\text{Content Width} = 200\text{px} - (10\text{px} \times 2)\ \text{(padding)} - (2\text{px} \times 2)\ \text{(border)} = \mathbf{176}\text{px}$$

- **Lợi ích vượt trội**: Điều này giúp việc tính toán bố cục dễ dàng và trực quan hơn rất nhiều, vì tổng kích thước của phần tử luôn luôn đúng với con số `width` và `height` mà bạn đã đặt trong mã nguồn. Đây là lý do tại sao nhiều khung làm việc (**framework**) CSS như Bootstrap, TailwindCSS và mọi dự án web hiện đại đều đặt `box-sizing: border-box;` cho tất cả các phần tử (thường là với bộ chọn toàn cục `* { box-sizing: border-box; }`).



1.3 Lưu Ý Chi Tiết Khi Sử Dụng `width` & `height`

Trong quá trình xây dựng giao diện thực tế, việc sử dụng `width` và `height` có thể dẫn đến nhiều lỗi hiển thị khó lường nếu không nắm chắc 5 nguyên lý cơ bản sau đây.

1.3.1 1. Giá Trị auto Và Hành Vi Khác Biệt Giữa Các Loại Phần Tử

a) Đối Với `width: auto;`

- **Phần tử khối (Block-level elements, mặc định `display: block`)**:
  - Sẽ **chiếm toàn bộ chiều rộng có sẵn** của phần tử cha (trừ đi `margin`, `padding`, `border` của chính nó).
  - Đây chính là lý do tại sao các thẻ `<div>`, `<p>`, `<section>` luôn tự động giãn ra chiếm hết chiều ngang dòng chữ dù nội dung của chúng rất ngắn.
- **Phần tử nội dòng (Inline-level elements, mặc định `display: inline`)**:
  - **Hoàn toàn KHÔNG nhận giá trị `width` cố định**. Mọi khai báo như `span { width: 300px; }` đều bị trình duyệt bỏ qua!
  - Chiều rộng của phần tử nội dòng luôn luôn được xác định duy nhất bởi độ dài văn bản hoặc kích thước phần tử con bên trong nó.
- **Phần tử nội dòng khối (`display: inline-block`) hoặc Phần tử trôi nổi (`float: left/right`)**:
  - Sẽ tự động co lại (**shrink-wrap**) để vừa khít với nội dung bên trong nó, trừ khi bạn chỉ định tường minh một giá trị `width` cụ thể.

b) Đối Với `height: auto;` (Rất Phổ Biến & Quan Trọng)

- Hầu hết các phần tử HTML đều có giá trị mặc định là `height: auto;`. Điều này có nghĩa là chiều cao của chúng sẽ tự động co giãn một cách hoàn hảo theo lượng nội dung bên trong.
- Đây là hành vi thường mong muốn nhất để tránh nội dung bị cắt cụt (khi kết hợp với `overflow: hidden`) hoặc xuất hiện thanh cuộn không đẹp mắt (khi dùng `overflow: auto/scroll`).

> **[Cảnh Báo] Cạm Bẫy Kinh Điển: Thuộc Tính height: 100% Không Hoạt Động!**
>
> Một phần tử con với khai báo `height: 100%;` sẽ **chỉ hoạt động nếu phần tử cha trực tiếp của nó có chiều cao được định nghĩa rõ ràng bằng một con số cụ thể** (ví dụ: `height: 500px;` hoặc `height: 100vh;`), chứ KHÔNG PHẢI là `height: auto;`.
> Nếu phần tử cha cũng để mặc định `height: auto;`, thì phần tử con sẽ **hoàn toàn không biết** 100% **của bao nhiêu** để tính toán, dẫn đến việc `height: 100%;` bị trình duyệt bỏ qua và phần tử con tự co lại theo nội dung của chính nó!

Ví Dụ Xử Lý Cạm Bẫy height: 100%

```
<!-- TRUONG HOP LOI: .child khong the dat 100% chieu cao vi .parent la auto -->
<div class="parent" style="height: auto;">
    <div class="child" style="height: 100%; background: lightcoral;">
        Con bi teo lai vi cha khong co chieu cao xac dinh!
    </div>
</div>

<!-- GIAI PHAP CHUAN: Xac dinh chieu cao cho cha hoac dung viewport unit -->
<div class="parent-fixed" style="height: 300px;">
    <div class="child" style="height: 100%; background: lightgreen;">
        Con nhan du 100</div></div>
```

1.3.2 2. Đơn Vị Tương Đối Và Đơn Vị Tuyệt Đối

a) Đơn Vị Tuyệt Đối (Absolute Units): `px`, `pt`, `cm`, `in`...

- Cố định, không thay đổi theo kích thước màn hình thiết bị hay kích cỡ phông chữ của người dùng.
- **Lưu ý thực chiến**: Việc lạm dụng và sử dụng `px` quá nhiều cho `width` và `height` của các container lớn có thể làm cho trang web trở nên cứng nhắc, kém linh hoạt và cực kỳ khó khăn khi chuyển đổi giao diện đáp ứng (**responsive**) trên màn hình điện thoại di động hoặc máy tính bảng.

b) Đơn Vị Tương Đối (Relative Units): `%`, `em`, `rem`, `vw`, `vh`...

- `%` **(Phần trăm)**: Luôn luôn tương đối so với kích thước của phần tử cha trực tiếp. Nếu phần tử cha không có kích thước rõ ràng hoặc đang để `height: auto;`, đơn vị `%` có thể không hoạt động như mong đợi.
- `em`: Tương đối so với `font-size` của chính phần tử đó (hoặc của phần tử cha nếu `font-size` không được định nghĩa trên chính nó). Rất hữu ích cho các thành phần UI có kích thước tỷ lệ thuận với văn bản (ví dụ: nút bấm có `padding` và `width` giãn nở nhịp nhàng theo cỡ chữ).
- `rem`: Tương đối so với `font-size` của phần tử gốc `<html>`. Giúp duy trì tỷ lệ kích thước hài hòa và đồng nhất trên toàn bộ hệ thống trang web.
- `vw` **(Viewport Width)** / `vh` **(Viewport Height)**: Tương đối so với kích thước thực của cửa sổ trình duyệt (**viewport**). 1vw là 1% chiều rộng viewport, 1vh là 1% chiều cao viewport. Đây là lựa chọn tuyệt vời cho các phần tử cần chiếm một tỷ lệ cố định của màn hình thiết bị, ví dụ: `height: 100vh;` để tạo một **Hero Section** tràn đầy trọn vẹn màn hình mở đầu.
- **Lưu ý**: Các đơn vị tương đối giúp tạo ra thiết kế linh hoạt, co giãn mượt mà và đáp ứng tốt hơn trên mọi thiết bị hiện đại.

1.3.3 3. Kiểm Soát Giới Hạn: `min-width`, `max-width`, `min-height`, `max-height`

Đây là nhóm bốn thuộc tính cực kỳ quan trọng và không thể thiếu trong kỹ thuật thiết kế giao diện đáp ứng (**Responsive Web Design**):

1. `max-width: 100%;` **(Quy chuẩn vàng tối thượng cho hình ảnh)**:
   - Giúp hình ảnh không bao giờ tràn ra khỏi vùng chứa của nó, tự động co lại một cách linh hoạt khi màn hình thu hẹp hơn kích thước gốc của ảnh.
   - Khối quy tắc kinh điển mà mọi lập trình viên Frontend đều phải thuộc lòng:

Bộ Quy Tắc Ảnh Responsive Chuẩn Công Nghiệp

```
img {
    max-width: 100%;
    height: auto;
    display: block;
}
```

   - Thuộc tính `height: auto;` đi kèm đảm bảo tỷ lệ khung hình (**aspect ratio**) của bức ảnh được bảo toàn nguyên vẹn, không bao giờ bị méo mó hình ảnh.

2. `min-width` / `min-height`:
   - Ngăn không cho một phần tử bị co lại quá nhỏ, đảm bảo người dùng luôn đọc được nội dung hoặc bấm trúng mục tiêu cảm ứng (**touch target**) trên điện thoại.
   - **Ví dụ**: `button { min-width: 120px; min-height: 40px; }` giúp nút bấm luôn đạt chuẩn tiếp cận (**Accessibility - a11y**).

3. `max-height`:
   - Cực kỳ hữu ích khi bạn muốn giới hạn chiều cao tối đa của một khối nội dung để tránh nó kéo dài trang web quá mức (ví dụ: khung bình luận, danh sách thông báo, hộp thoại điều khoản).
   - Thường được kết hợp với `overflow-y: auto;` để sinh ra thanh cuộn dọc mượt mà khi lượng nội dung vượt quá giới hạn chiều cao cho phép.
   - **Ví dụ**: `max-height: 300px; overflow-y: auto;`.

1.3.4 4. Mối Tương Quan & Ảnh Hưởng Từ `display` Và `position`

Hành vi của `width` và `height` phụ thuộc mật thiết vào cách phần tử được định kiểu hiển thị và định vị:

1. `display: inline;`:
   - Các phần tử nội dòng (như `<a>`, `<span>`, `<strong>`, `<em>`) sẽ **hoàn toàn không nhận** các thuộc tính `width`, `height`, `margin-top`, `margin-bottom`, `padding-top`, và `padding-bottom` theo cách thông thường.
   - Chiều rộng và chiều cao của chúng được xác định hoàn toàn bởi dung lượng chữ bên trong.

2. `display: inline-block;`:
   - Các phần tử này thừa hưởng khả năng thiết lập `width` và `height` độc lập như một khối (**block**), nhưng đồng thời vẫn nằm thẳng hàng trên cùng một dòng với các phần tử nội dòng khác.

3. `position: absolute;` / `position: fixed;`:
   - Khi một phần tử được định vị tuyệt đối hoặc cố định, nó sẽ **bị rút hoàn toàn khỏi dòng chảy tài liệu thông thường** (**normal document flow**).

- `width` và `height` của chúng có thể được thiết lập độc lập hoặc có thể dùng đồng thời cặp thuộc tính vị trí `top: 0; bottom: 0;` và `left: 0; right: 0;` để tự động kéo giãn phủ kín không gian.
- **Ví dụ điển hình**: Khai báo `width: 100%; height: 100%; position: absolute; top: 0; left: 0;` sẽ làm cho lớp phủ (**overlay**) lấp đầy toàn bộ phần tử cha gần nhất có khai báo `position` khác `static`.

1.3.5 5. Tương Tác Giữa `overflow` Và Kích Thước `width` / `height`

Khi bạn thiết lập `width` hoặc `height` cố định cho một phần tử và khối lượng nội dung bên trong lại lớn hơn không gian kích thước đó, thuộc tính `overflow` sẽ quyết định cách thức trình duyệt xử lý phần nội dung bị tràn:

- `overflow: visible;` (**Mặc định**): Nội dung bị tràn sẽ không bị cắt bỏ mà tràn ra ngoài đường biên của phần tử, đè lên các khối giao diện khác ở phía dưới.
- `overflow: hidden;`: Phần nội dung bị tràn ra ngoài kích thước hộp sẽ bị cắt bỏ thẳng tay và hoàn toàn ẩn đi, người dùng không thể xem được.
- `overflow: scroll;`: Luôn luôn hiển thị thanh cuộn (dù nội dung có bị tràn hay không), cho phép người dùng cuộn xem toàn bộ dữ liệu.
- `overflow: auto;`: Trình duyệt thông minh chỉ hiển thị thanh cuộn khi nội dung thực sự bị tràn vượt quá giới hạn của hộp chứa.

1.3.6 6. Lời Khuyên Quan Trọng & Thiết Lập Reset Toàn Cục Chuẩn

[Mẹo] Quy Tắc Vàng Khởi Đầu Cho Mọi Dự Án Web Hiện Đại

Trong hầu hết mọi dự án web hiện đại (từ website cá nhân cho tới các ứng dụng doanh nghiệp quy mô lớn), bạn nên **luôn luôn bắt đầu bằng việc thiết lập `box-sizing: border-box;` cho tất cả các phần tử** trong tệp tin CSS reset hoặc stylesheet toàn cục:

```
*, *::before, *::after {
    box-sizing: border-box;
}
```

Lợi ích thực tế: Thiết lập này sẽ giúp bạn quản lý kích thước các phần tử một cách trực quan và dễ dàng hơn rất nhiều. Mọi con số `width` và `height` bạn đặt ra sẽ luôn đúng tuyệt đối với diện tích chiếm chỗ trên màn hình, loại bỏ vĩnh viễn những lỗi tính toán sai lệch kích thước gây vỡ bố cục khi thêm `padding` và `border`.